1. Add Spring Cloud Config Client to Edge Service to make it fetch configuration data from Config Service.
2. Configure the Cloud Native Buildpacks integration, containerize the application, and define the commit stage of the deployment pipeline, as you learned in chapters 3 and 6.
3. Write the Deployment and Service manifests for deploying Edge Service to a Kubernetes cluster.
4. Configure Tilt to automate the Edge Service deployment to your local Kubernetes cluster initialized with minikube.

You can refer to the Chapter09/09-end folder in the code repository accompanying the book to check the final result (https://github.com/ThomasVitale/cloud-native-spring-in-action). You can also deploy the backing services from the manifests available in the Chapter09/09-end/polar-deployment/kubernetes/platform/development folder with `kubectl apply -f services`.

9.5 Managing external access with Kubernetes Ingress

Spring Cloud Gateway helps you define an edge service where you can implement several patterns and cross-cutting concerns at the ingress point of a system. In the previous sections, you saw how to use it as an API gateway, implement resilience patterns like rate limiting and circuit breakers, and define distributed sessions. In chapters 11 and 12, we'll also add authentication and authorization features to Edge Service.

Edge Service represents the entry point to the Polar Bookshop system. However, when it's deployed in a Kubernetes cluster, it's only accessible from within the cluster itself. In chapter 7, we used the *port-forward* feature to expose a Kubernetes Service defined in a minikube cluster to your local computer. That's a useful strategy during development, but it's not suitable for production.